Høgskulen på Vestlandet

DAT107 - Databaser

Video 14: NoSQL – Distribuerte databaser

Introduksjon til NoSQL

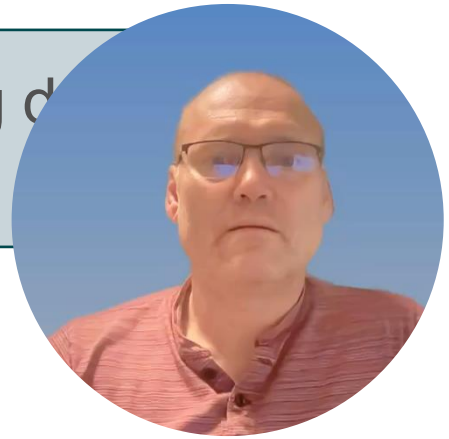
Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

Kjenne til hva som kjennetegner **distribuerte databaser**, samt kjenne til begrepene **kluster, vertikal- og horisontal skalering, sharding og replikering**.

Kjenne til **fordeler og ulemper** med distribuerte databaser.

Kjenne til og forklare **CAP-teoremet,** og hvilken betydning d
har for ulike database systemer.



Litteratur

Læreboka: Kapittel 15.3.1 og 15.3.2

Foilene

https://www.ibm.com/topics/cap-theorem
(Ikke siste del «Microservices and the CAP theorem»

En-node (sentralisert) vs. kluster (distributert)

- ✓ **En database instans kjører på en enkelt maskin**
- ✓ Vanligste formen for relasjons databaser
- ✓ Begrenset hvor mye man kan skalere en maskin (CPU og minne)
  - Disk (SAN) er unntaket
- ✓ Partisjonering

- ✓ **Kluster: Flere maskiner/noder utgjør et database system (en database)**
- ✓ **Ikke-delt** Arkitektur (En: Shared-nothing)
  - Hver maskin/ node har selvstendig lagring av data
  - Svært skalerbar
- ✓ **Delt-Disk** Arkitektur (En: Shared-Disk)
  - Skalere antall CPU-er og Minne
  - Mye mindre skalerbar
  - Eks: Oracle RAC
- ✓ **Sharding (partisjonering)**
  - Dele opp en stor databas
    partisjoner (som lagres på

Vertical Scaling vs. Horizontal Scaling

- ✓ Øke tilgjengelige ressurser på en server
  - Mer CPU
  - Mer minne
  - Mer disk (lagringskapasitet)
- ✓ Vanligste formen for skalering for relasjons databaser
- ✓ Begrenset hvor mye man kan skalere en maskin
- ✓ Kommersielle (DB) lisenser er ofte knyttet til antall CPU-er (eller kjerner)
- ✓ Dyrt å skalere i det uendelige

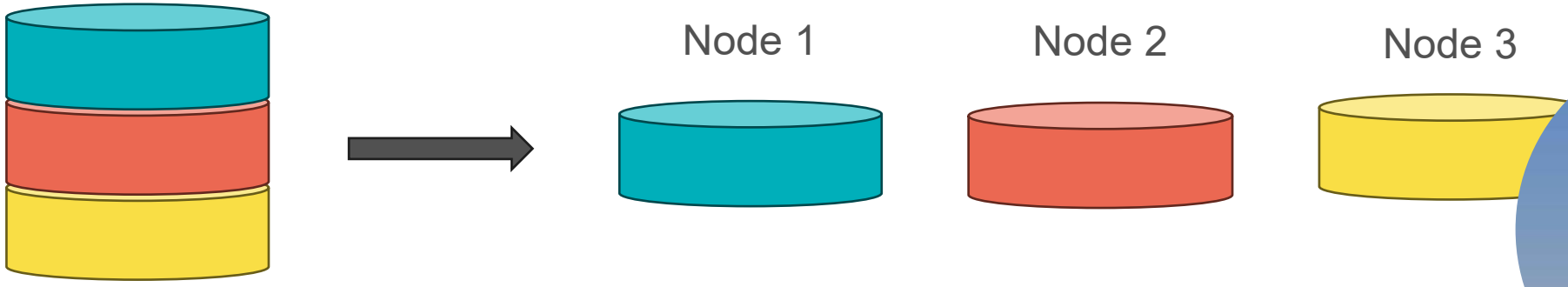
- ✓ Legge til flere maskiner eller noder til et system
- ✓ NoSQL databaser lar deg raskt skalere horisontalt (dvs legge til flere database noder i et kluster)

Sharding

Distribuerte databaser

Høgskulen på Vestlandet

- ✓ Distribuere et dataset ut over flere noder / datamaskiner
  - **Horisontal skalering** (Horizontal scaling)
  - **Overhead og kompleksitet** i å sette opp shards, vedlikeholde dataene på hvert shard, og riktig ruting av forespørsler på tvers av disse shards
  - **Partisjons mapping:** Benytter hashing algoritmer for å avgjøre på hvilken partisjon (node data ligger)

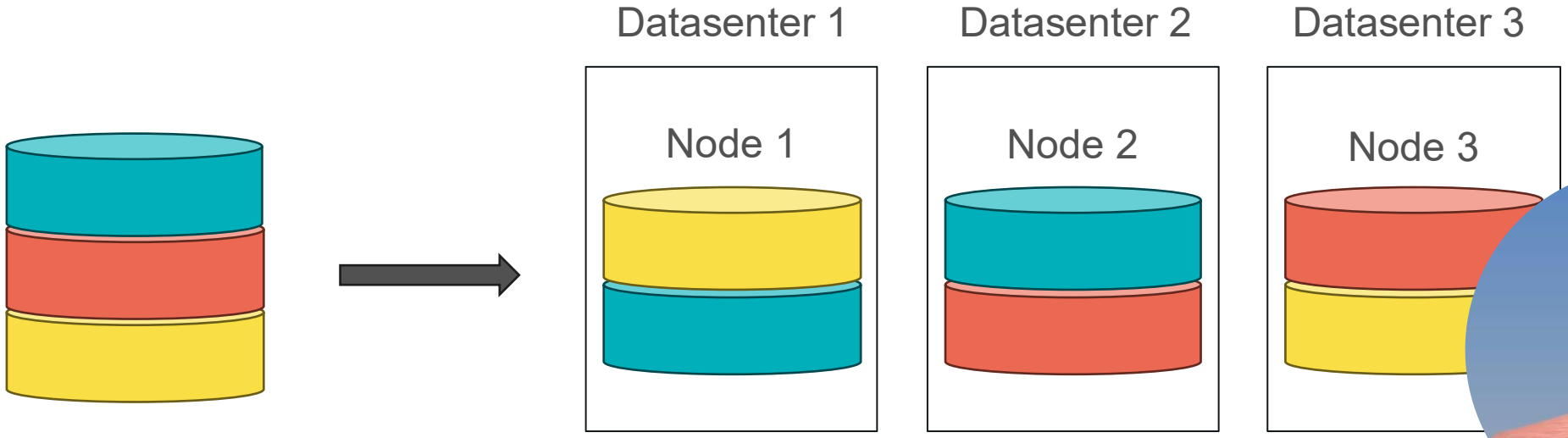


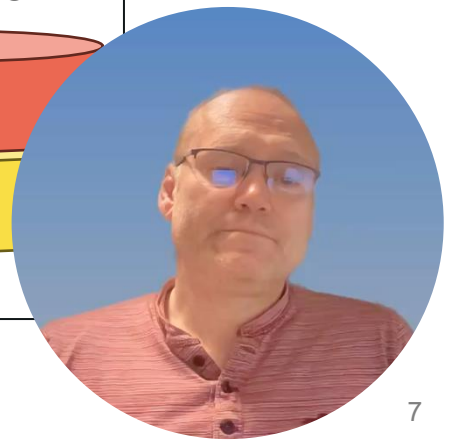
Replikering

Distribuerte databaser

Høgskulen på Vestlandet

- ✓ Kopi av data på flere noder
  - Økt tilgjengelighet
  - Bedre ytelse for les (kan lese fra flere steder, last balansering)
  - Hva med skriv (write) og konsistens?





Fordeler

Distribuerte databaser

Høgskulen på Vestlandet

- ✓ **Last balansering**: Spre lasten over flere noder/ ressurser
- ✓ **Skalering**: Kunne serve et større antall brukere (samtidig) (og mere data)
- ✓ **Data redundans**: Sørge for at man har mer en en kopi av data
- ✓ **Tilgjengelighet**: Unngå «single-point of failure»
- ✓ **Vedlikehold:** Kan oppgradere en og en server (uten nedetid).

  (NB! Hører vel egentlig til under tilgjengelighet)

Ulemper

Distribuerte databaser

Høgskulen på Vestlandet

- ✓ **Kompleksitet**
  - Klusteroppsett kan være komplekst å **konfigurere og administrere**. Det krever ofte spesialisert kunnskap for å sette opp og vedlikeholde, noe som kan være en utfordring for team uten erfaring med distribuerte systemer.
- ✓ **Datareplikering og in-konsistens**:
  - Mange brukere jobber mot samme data spred over flere noder
  - Replikering av data mellom noder kan føre til inkonsistens, spesielt i systemer som bruker eventual consistency. Dette kan skape utfordringer med datakvalitet og pålitelighet.
- ✓ **Ytelse**
  - I noen tilfeller **kan** ytelsen bli **påvirket av nettverkslatens mellom noder**, spesielt hvis hentes fra flere noder for å oppfylle en forespørsel. Dette kan føre til tregere responstide

CAP teorem

En teori om det «mulige»

Høgskulen på Vestlandet

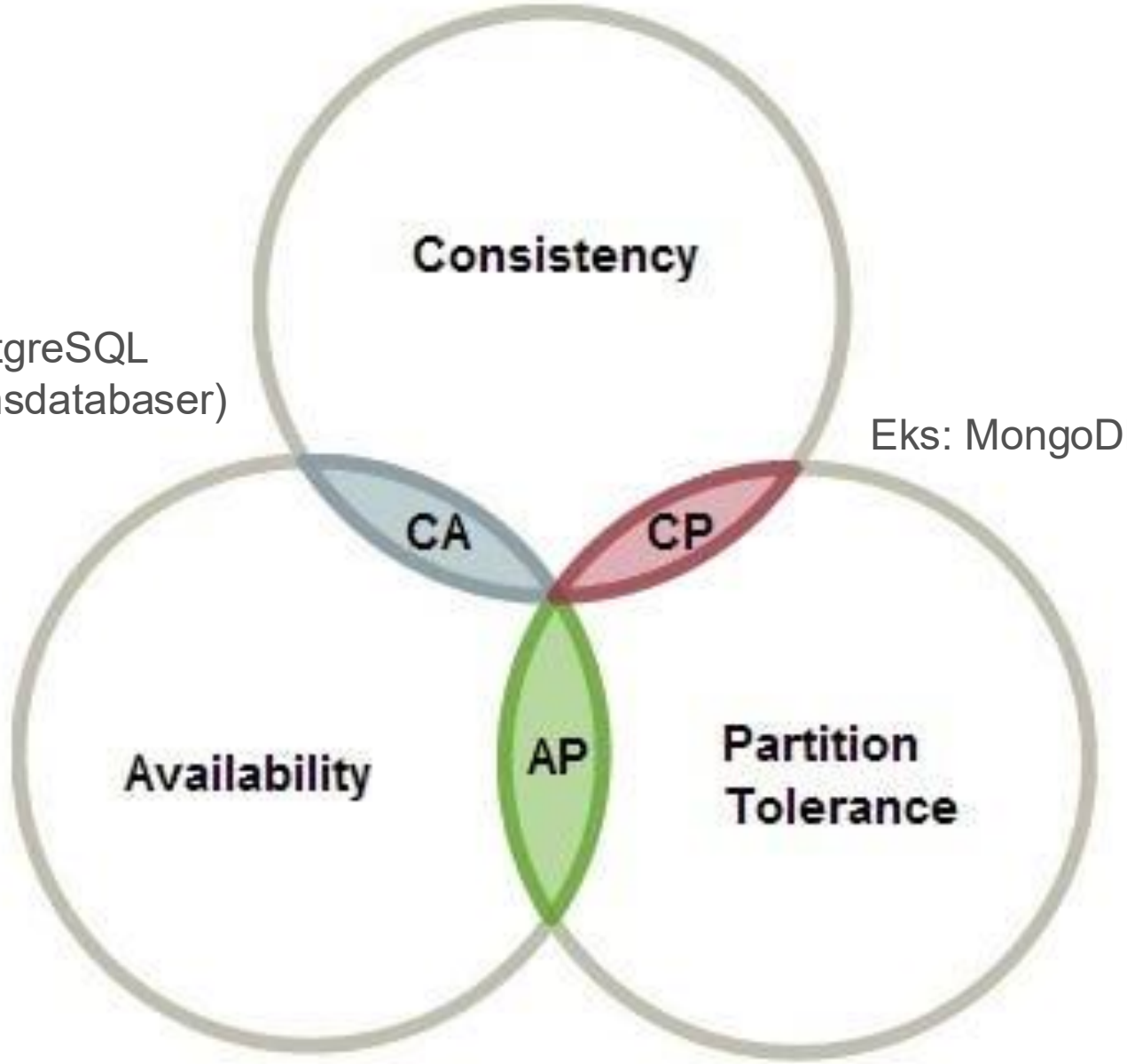
- ✓ Først lansert i 1998. Presentert av Eric Brewer under en konferanse i 2000 angående distribuert databehandling
- ✓ Teoremet går ut på at man kan **KUN velge 2 av de følgende 3 egenskaper** når man designer et distribuert system.
  - **Konsistens** (en: Consistency)
    - Alle klienter ser samme data på samme tid, uansett hvilken node de er koblet til. (Når data skrives må den umiddelbart replikeres til alle andre noder før skriving kan anses som "fullført")
  - **Tilgjengelighet** (en: Availability)
    - Enhver klient som sender en forespørsel om data får svar, selv om en eller flere noder er nede
  - **Partisjon toleranse** (en: Partition tolerance)
    - "Partisjon" = kommuikasjons brudd mellom noder i et distribuert system
    - Et kluster må kunne fungere videre uavhengig av kommunikasjons pro mellom noder i et system

Høgskulen på Vestlandet

Når det gjelder moderne distributerte systemer, så er Partisjons Toleranse ikke en opsjon – men en nødvendighet. Derfor, må slike systemer velge mellom konsistens (Consistency) og tilgjengelighet (Availabiltiy).

Høgskulen på Vestlandet

Eks: PostgreSQL
(Relasjonsdatabaser)

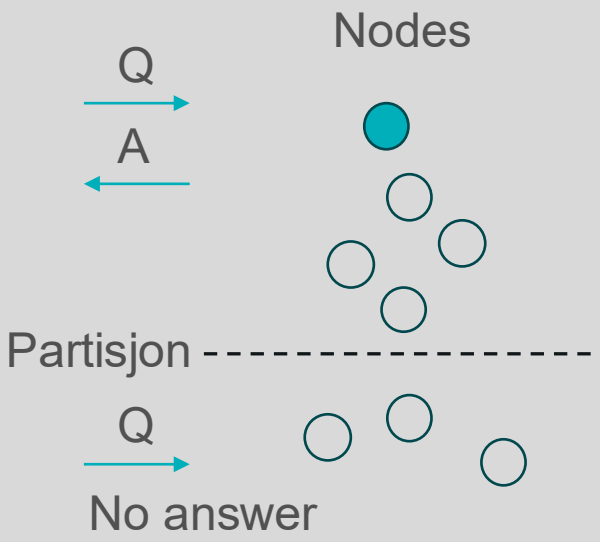


Eks: MongoDB

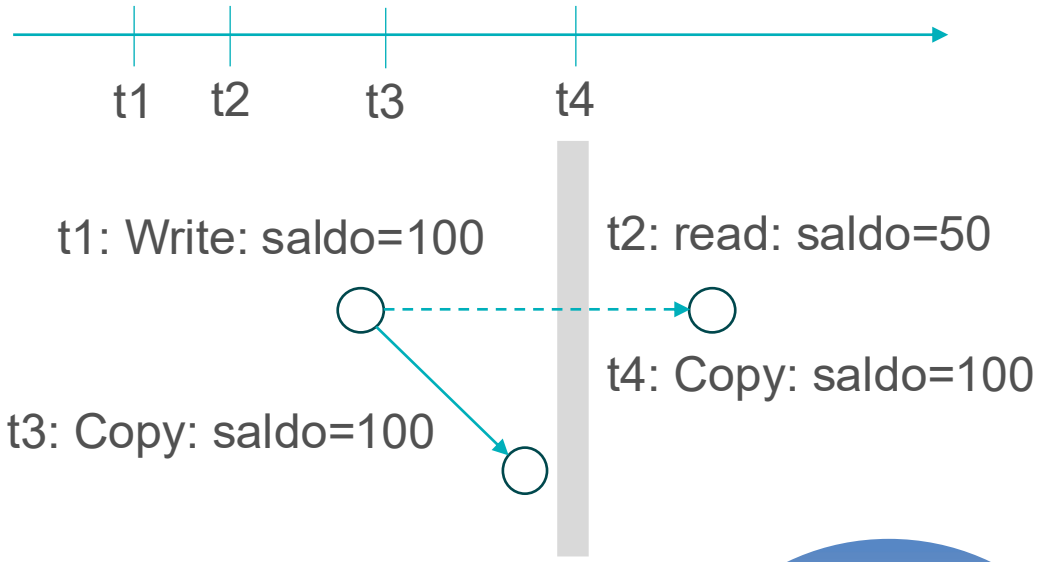
Eks: Cassandra, DynamoDB

Høgskulen på Vestlandet

CP design



AP design



Relasjonsdatabaser har (stort sett)
CA design

Ulike egenskaper

NoSQL Databaser

Høgskulen på Vestlandet

- ✓ Først lansert i 1998. Presentert av Eric Brewer under en konferanse i 2000 angående distribuert databehandling
- ✓ Teoremet går ut på at man kan **KUN velge 2 av de følgende 3 egenskaper** når man designer et distribuert system.
  - **Konsistens** (en: Consistency)
    - Alle klienter ser samme data på samme tid, uansett hvilken node de er koblet til. (Når data skrives må den umiddelbart replikeres til alle andre noder før skriving kan anses som "fullført")
  - **Tilgjengelighet** (en: Availability)
    - Enhver klient som sender en forespørsel om data får svar, selv om en eller flere noder er nede
  - **Partisjon toleranse** (en: Partition tolerance)
    - "Partisjon" = kommuikasjons brudd mellom noder i et distribuert system
    - Et kluster må kunne fungere videre uavhengig av kommunikasjons pro mellom noder i et system

Ulike egenskaper

NoSQL Databaser

Høgskulen på Vestlandet

- ✓ MongoDB
  - **Eksempel på CP design** (Konsistens og Partisjons toleranse)
  - Single-master (primær node): Mottar alle «write» operasjoner
  - Sekundær noder: Replikerer primær nodens operasjons log
  - Når primær noden blir utilgjengelig vil den sekundære noden med den nyeste operasjonsloggen bli valgt som ny primær node. Databasen er **ikke tilgjengelig** før alle sekundærnodene har tatt igjen den nye primærnoden.
- ✓ Cassandra og Amazon DynamoDB
  - **Eksempel på AP design** (Availabilty og Partisjons toleranse)
  - Cassandra (Ingen master node), mens DynamoDB (master)
  - Kan ikke alltid gi konsistens, men **«eventuell konsistens»** (en: eventual co
    - Klienter kan skrive til alle noder når som helst og avstemme inkonsekv raskt som mulig (når partisjon blir «frisk»)

Oppsummering

Distribuerte Databaser

Høgskulen på Vestlandet

- ✓ Sentralisert (en-node) vs. distribuerte databaser (kluster)
- ✓ Vertikal vs. Horisontal skalering
- ✓ Sharding & Replikering
- ✓ Fordeler og ulemper med distribuerte databaser
- ✓ CAP-teorem
- ✓ Partisjons mapping: Hashing algoritme

DEMO